

o_que_e_shellscript

O que é linguagem shellscript

1. INDEX

1. Introdução

1. [Objetivo.](#)
2. [Pre-requisitos.](#)
3. [benefícios.](#)

2. [Descrição.](#)

1. [Anotações](#)
2. [Functions](#)

3. [Exemplos.](#)

1. [Functions](#)

4. [Conteúdo01](#)

5. [Referências.](#)

6. [Histórico.](#)

2. CONTEÚDO

1. Introdução

1. **Objetivo:**

1. ShellScript é a linguagem usada pelo linux para automatizar processos.
2. 

2. **Pre-requisitos:**

1. Uso de um shell Unix / Linux interativo;
2. Conhecimento mínimo de programação - o uso de variáveis, funções, é um conhecimento básico útil;
3. Compreensão de alguns comandos Unix / Linux e competência no uso de alguns dos mais comuns . (ls , cp , echo , etc);
4. Programadores de ruby , perl , python , C , Pascal ou qualquer linguagem de programação (mesmo BASIC) que podem ler scripts de shell, mas não sentem que entendem exatamente como funcionam..
5. 

3. **Benefícios:**

1. Automatizar processos do sistema operacional linux.

2. 

2. Descrição

1. Anotações de meus estudos

1. As **passagem de parâmetros** e o **retorno das funções** em shellscrip é totalmente diferente de todas as linguagens que conheço. Veja abaixo a [descrição de funções](#) e os [exemplos](#) para melhor compreensão.

2. O comando **exit** também é diferente, ao invés de sair da função, ele aborta a função (igual a **halt** do pascal). Veja os exemplos abaixo.

3. 

2. As funções em shellscrip tem a sintaxe **nomeDaFunção(){}** e podem retorna valor ou não da seguinte maneira:

1. Use o comando **exit** para terminar o script e retorna ao script que o chamou (igual ao halt do pascal);

2. Use o comando **return** para encerrar a função e retornar o valor numérico (não aceita string) fornecido para a seção de chamada do script;

1. **Obs:** Para capturar um retorno de uma função é totalmente diferente de qualquer linguagem que eu conhecia. Usa-se a sequência de caractere **\$?** após a execução da função.

3. Alterar o estado de uma variável ou variáveis globais;

1. Nota: A função shell não pode alterar seus parâmetros e sim parâmetros globais.

4. Saída de eco para stdout, que será capturada pelo chamador assim que c = **expr \$a + \$b** for capturada.

3. 

3. Exemplos.

1. Exemplos de funções:

1. Este código ShellScript mostra o uso das função **expr**, **return** e resultado da função em **\$?**.

```
# Script subtrai.sh
#!/bin/sh

# função subtrai
subtrai(){
    param1=$1
    param2=$2
    echo parâmetro 01 = $param1
    echo parâmetro 02 = $param2

    echo $param1
    echo $param2
    result=`expr $param1 - $param2`
    return $result
}

# parte principal do programa
subtrai 5 2
result="$?"
# A sequencia $? contém o valor de return.
echo "O valor retorna da expressão subtrai 5 2 =
$result"
```

2. Este código ShellScript mostra o uso das função **expr**, **exit** e resultado da função em **\$?**.

```
# Script soma.sh
#!/bin/sh

# função soma
soma(){
    param1=$1
    param2=$2
    echo parâmetro 01 = $param1
    echo parâmetro 02 = $param2

    echo $param1
    echo $param2
    result=`expr $param1 + $param2`
    exit $result
}

# parte principal do programa
soma 1 2
# A sequencia $? contém o valor de exit mais nunca
imprimira
# a linha abaixo aqui porque o comando exit aborta o
script.
echo "O valor retorna da expressão soma 1 2 = $?"
```

```
# Script testsoma.sh
#!/bin/sh

# teste da chamada de função estando em outro arquivo.
./soma.sh
echo "0 valor retorna da expressão soma 1 2 = $?"
```

3. 

2. item 02.

3. 

4. REFERÊNCIAS

1. [Manual do usuário](#)
2. [ShellScript functions](#)
3. [Livro inacabado do comando sed](#)
4. <#>
5. <#>
6. 

5. HISTÓRICO

1.02/06/2021

◦ 

2.02/06/2021

- ☒ Criar este documento baseado no modelo02.md ;
- ☐ Escrever tópico Objetivos;
- ☐ Escrever tópico Pre-requisitos
- ☐ Escrever tópico Benefícios
- ☐ Escrever tópico Descrição
- ☐ Escrever tópico Exemplos
- ☐ Escrever tópico Conteúdo 01
- ☐ Escrever tópico Referências
- ☐ Atualizar o histórico deste documento.
- ☐ Testar este documento depois após uma semana de concluído.

